Lab 3 — N-Grams

Name: Omar Hejazi
Student ID: 2230009001
Dataset: Large Random Tweets from Pakistan

Task agenda

1. Load the data (`adizafar/large-random-tweets-from-pakistan`).
2. Preprocess: remove hashtags, `RT` , websites, mentions and emojis.
3. Build an MLE model with n-grams (bigram).
4. Evaluate the model.
5. Calculate the probability of the bigram *(pakistan is)*.
6. Calculate the perplexity of the word *(pakistan)*.

In [1]:
```
import random
import re
from collections import Counter
from pathlib import Path

import matplotlib.pyplot as plt
import nltk
import pandas as pd

for pkg in ["punkt", "punkt_tab"]:
    nltk.download(pkg, quiet=True)

DATA_DIR = Path("data")
SEED = 42
random.seed(SEED)
```

1. Loading the data

In [2]:
```
csv_path = DATA_DIR / "Random Tweets from Pakistan- Cleaned- Anonymous.csv"

if not csv_path.exists():
    import kagglehub

    cache = kagglehub.dataset_download("adizafar/large-random-tweets-from-pakistan")
    csv_path = next(Path(cache).glob("*.csv"))

raw = pd.read_csv(csv_path, encoding="utf-8", encoding_errors="replace", low_memory=False)
print(raw.shape)
raw.columns.tolist()
```

```
(202202, 7)
```

Out[2]:
```
['Unnamed: 0',
 'created_at_tweet',
 'full_text',
 'retweet_count',
 'favorite_count',
 'reply_count',
 'location']
```

The file does not decode cleanly as UTF-8 — a handful of rows contain truncated multi-byte sequences. `encoding_errors="replace"` substitutes those bytes with `U+FFFD` rather than aborting the read; the affected characters are dropped later anyway when the text is reduced to ASCII letters.

In [3]:
```
tweets = raw["full_text"].dropna().astype(str)
print(f"Tweets with text: {len(tweets):,}")
for t in tweets.head(4):
    print("-", t[:110])
```

```
Tweets with text: 202,151
- @MaryamNSharif @Tanverhussan @SenPervaiz تیرا لیڈر میرا لیڈر نواز شریف ❤️نواز شریف کیا آپکا بھی ھے؟؟؟
https:/
- Happy birthday to my brother n boss , May you have many many more🎂🎂🎂, you are Gem 💎
#HappyBirthdayAtifRauf
- ❤️❤️
- `suspicious °jikook au jimin'in yaşadığı kasabada aniden başlayan cinayetler, onun kendisini
sürekli takip ede
```

The corpus is bilingual: a large share of the tweets are in Urdu. A bigram model trained on both languages at once would spend its probability mass modelling the boundary between two grammars it cannot mix, and the generated output would be unreadable. Since the task is to generate English tweets, I filter to the predominantly-Latin-script tweets first.

In [4]:
```
def latin_ratio(text):
    letters = [c for c in text if c.isalpha()]
    if not letters:
        return 0.0
    return sum(c.isascii() for c in letters) / len(letters)


english = tweets[tweets.map(latin_ratio) > 0.9]
print(f"Predominantly Latin-script tweets: {len(english):,} ({len(english) /
len(tweets):.0%})")
```

```
Predominantly Latin-script tweets: 161,471 (80%)
```

This is a *script* filter, not a language filter, and the distinction shows up in the cleaned output below: romanised Urdu ("jb tm moon lover ho gy to sooraj to tappy ga na") and even some Turkish are written in Latin characters and survive the filter. Doing this properly would need language identification — `langdetect` or `fasttext` — which is beyond this lab. The residual non-English text is a known limitation of the model built here, and it is part of why the generated tweets in section 3 occasionally switch languages mid-sentence.

---

2. Preprocessing

The task lists five things to strip. Each one is a token type that would otherwise pollute the model's vocabulary with strings it can never usefully predict:

Remove	Why
Hashtags	topic markers, not part of the sentence grammar
`RT`	a retweet marker, not a word
URLs	every link is unique, so each becomes a one-off vocabulary entry
Mentions	`@username` is an address, and there are tens of thousands of them
Emojis	not tokens the model can pronounce back into a sentence

In [5]:
```
try:
    import emoji
except ImportError:
    !pip install -q emoji
    import emoji

URL = re.compile(r"https?://\S+|www\.\S+")
MENTION = re.compile(r"@\w+")
HASHTAG = re.compile(r"#\w+")
RT = re.compile(r"^\s*RT\b:?|\bRT\b")
AMP = re.compile(r"&(amp|lt|gt|quot|#\d+);")
NON_TEXT = re.compile(r"[^a-z0-9'.!? ]+")
SPACES = re.compile(r"\s+")


def clean_tweet(text):
    text = URL.sub(" ", text)
    text = MENTION.sub(" ", text)
    text = HASHTAG.sub(" ", text)
    text = RT.sub(" ", text)
    text = emoji.replace_emoji(text, replace=" ")
    text = AMP.sub(" ", text)
    text = text.lower()
    text = NON_TEXT.sub(" ", text)
    return SPACES.sub(" ", text).strip()
```

Sentence-ending punctuation (`.`, `!`, `?`) and the apostrophe survive `NON_TEXT`. The apostrophe keeps contractions like `don't` in one piece, and the terminators are kept so the text can be split into sentences — a bigram model needs sentence boundaries to know where `<s>` and `</s>` go.

In [6]:
```
demo = "RT @ImranKhanPTI: Pakistan is winning today PK🔥 #PakvsInd watch here
https://t.co/abc123"
print("raw:    ", demo)
print("cleaned:", clean_tweet(demo))
```

```
raw:     RT @ImranKhanPTI: Pakistan is winning today PK🔥 #PakvsInd watch here
https://t.co/abc123
cleaned: pakistan is winning today watch here
```

In [7]:
```
cleaned = english.map(clean_tweet)
cleaned = cleaned[cleaned.str.split().str.len() >= 4]  # too short to contain a usable
bigram
print(f"Tweets remaining after cleaning: {len(cleaned):,}")
cleaned.head(8).tolist()
```

```
Tweets remaining after cleaning: 115,073
```

Out[7]:
```
['happy birthday to my brother n boss may you have many many more you are gem',
 "suspicious jikook au jimin'in ya ad kasabada aniden ba layan cinayetler onun kendisini s
rekli takip eden yeni kom usu jungkook'tan phelenmesine neden olur",
 'alexa skip to 2021',
 'jb tm moon lover ho gy to sooraj to tappy ga na',
 'alexa skip to 2022 now',
 'but this was one of my first shoots about 6 years ago',
 'trying phoneix in 140 ping',
 'cant sleep so why not .']
```

From tweets to tokenised sentences

`padded_everygram_pipeline` expects a list of token lists — one per sentence. Tweets often contain more than one sentence, so I split them first rather than treating a whole tweet as a single sequence; that keeps `</s>` attached to real sentence ends.

In [8]:
```python
from nltk.tokenize import sent_tokenize, word_tokenize

corpus = []
for text in cleaned:
    for sentence in sent_tokenize(text):
        tokens = [t for t in word_tokenize(sentence) if t not in {".", "!", "?"}]
        if len(tokens) >= 3:
            corpus.append(tokens)

print(f"Sentences: {len(corpus):,}")
print(f"Tokens:    {sum(len(s) for s in corpus):,}")
print(f"Vocabulary:{len(set(t for s in corpus for t in s)):,}")
corpus[:3]
```

```
Sentences: 163,580
Tokens:    2,136,267
Vocabulary:78,083
```

Out[8]:
```
[['happy',
  'birthday',
  'to',
  'my',
  'brother',
  'n',
  'boss',
  'may',
  'you',
  'have',
  'many',
  'many',
  'more',
  'you',
  'are',
  'gem'],
 ['suspicious',
  'jikook',
  'au',
  "jimin'in",
  'ya',
  'ad',
  'kasabada',
  'aniden',
  'ba',
  'layan',
  'cinayetler',
  'onun',
  'kendisini',
  's',
  'rekli',
  'takip',
  'eden',
  'yeni',
  'kom',
  'usu',
  "jungkook'tan",
  'phelenmesine',
  'neden',
  'olur'],
 ['alexa', 'skip', 'to', '2021']]
```

Held-out split

Evaluating a language model on the text it was trained on is meaningless — an MLE model memorises its training set, so it would score perfectly. I hold out 10% of the sentences before fitting.

In [9]:
```
random.shuffle(corpus)
split = int(0.9 * len(corpus))
train_sents, test_sents = corpus[:split], corpus[split:]
print(f"train: {len(train_sents):,} sentences   test: {len(test_sents):,} sentences")
```

```
train: 147,222 sentences   test: 16,358 sentences
```

---

3. Building the bigram MLE model

`padded_everygram_pipeline(2, sentences)` returns two generators: the padded n-grams for every sentence (unigrams *and* bigrams — hence "everygram"), and the flat vocabulary stream. Both are consumed by `fit`, and because they are generators they can only be consumed once, which is why the pipeline is rebuilt whenever a new model is trained.

In [10]:
```
from nltk.lm import MLE
from nltk.lm.preprocessing import padded_everygram_pipeline

N = 2
train_data, vocab = padded_everygram_pipeline(N, train_sents)

lm = MLE(N)
lm.fit(train_data, vocab)

print(f"Model order:      {lm.order}")
print(f"Vocabulary size:  {len(lm.vocab):,}")
print(f"Unigram 'pakistan' count: {lm.counts['pakistan']:,}")
print(f"Bigram  ('pakistan', 'is') count: {lm.counts[['pakistan']]['is']:,}")
```

```
Model order:      2
Vocabulary size:  73,808
Unigram 'pakistan' count: 12,150
Bigram  ('pakistan', 'is') count: 616
```

Generating tweets

`lm.generate` samples from the model one token at a time. The raw output is a token list, so I detokenise it back into readable text and stop at the `</s>` marker.

In [11]:
```python
from nltk.tokenize.treebank import TreebankWordDetokenizer

detokenize = TreebankWordDetokenizer().detokenize


def generate_tweet(model, num_words=18, seed_text=None, random_seed=SEED):
    tokens = []
    for token in model.generate(num_words, text_seed=seed_text, random_seed=random_seed):
        if token == "</s>":
            break
        if token == "<s>":
            continue
        tokens.append(token)
    text = detokenize(tokens)
    if seed_text:
        text = f"{' '.join(seed_text)} {text}"
    return text


def sample_tweets(model, n, seed_text=None, start=SEED):
    # The model can emit </s> immediately, which yields an empty string; skip those.
    out, offset = [], 0
    while len(out) < n:
        text = generate_tweet(model, seed_text=seed_text, random_seed=start + offset)
        offset += 1
        if text.strip():
            out.append(text)
    return out


for i, text in enumerate(sample_tweets(lm, 8), 1):
    print(f"{i}. {text}")
```

```
1. no 9 august 13 month limit to attract investment
2. genuine issues that cpec and
3. binance customer care but arsalan's entire city during the almighty
4. tk koi moka nai but my new zealand 87.8
5. douchebag ...
6. khan
7. inky dekhny ko case is when
8. august 2021
```

In [12]:
```python
for i, text in enumerate(sample_tweets(lm, 5, seed_text=["pakistan"]), 1):
    print(f"{i}. {text}")
```

```
1. pakistan just 30 a.m. skin in rose water
2. pakistan 's play a living in the kashmiris as a la ligne un ke roo pary and the sort.quote
3. pakistan cricket journalist siddique jaan
4. pakistan and let's forum
5. pakistan team.may punjabi leader who called on pizza will always a gol matol maryam
aurangzeb slowly include ur words
```

The output is locally plausible and globally incoherent, which is exactly what a bigram model should produce. Each word is a reasonable successor to the one before it, because that is the only thing the model conditions on; two words back, the context is already gone. Anything resembling a sentence-level idea is coincidence.

Raising `n` would extend the context window, but with a fixed corpus it also makes the counts sparser — the usual bias/variance trade-off for n-gram models.

4. Evaluating the model

Why plain MLE perplexity is infinite

MLE assigns probability strictly proportional to counts, so any bigram that did not occur in training gets probability zero. A single unseen bigram in the test set drives the whole perplexity to infinity.

In [13]:
```
from nltk.util import ngrams
from nltk.lm.preprocessing import pad_both_ends

test_bigrams = [
    bg
    for sent in test_sents
    for bg in ngrams(pad_both_ends(sent, n=N), n=N)
]
print(f"Bigrams in the test set: {len(test_bigrams):,}")

unseen = sum(1 for bg in test_bigrams if lm.score(bg[-1], bg[:-1]) == 0)
print(f"Assigned zero probability: {unseen:,} ({unseen / len(test_bigrams):.1%})")
print(f"\nMLE perplexity on held-out data: {lm.perplexity(test_bigrams)}")
```

```
Bigrams in the test set: 230,998
Assigned zero probability: 48,759 (21.1%)
MLE perplexity on held-out data: inf
```

That `inf` is not a bug — it is the defining weakness of the maximum-likelihood estimator, and the reason smoothing exists. The model is not merely uncertain about the unseen bigrams; it claims they are impossible.

Note that this never shows up if you evaluate on the training data, which is why the split above mattered:

In [14]:
```
train_sample = [
    bg for sent in train_sents[:2000] for bg in ngrams(pad_both_ends(sent, n=N), n=N)
]
print(f"MLE perplexity on training data: {lm.perplexity(train_sample):,.1f}")
```

```
MLE perplexity on training data: 63.1
```

Laplace smoothing

`Laplace` adds one to every count, so no bigram has zero probability and perplexity becomes finite and comparable.

In [15]:
```
from nltk.lm import Laplace

train_data, vocab = padded_everygram_pipeline(N, train_sents)
laplace = Laplace(N)
laplace.fit(train_data, vocab)

print(f"Laplace perplexity on held-out data: {laplace.perplexity(test_bigrams):,.1f}")
print(f"Vocabulary size:                     {len(laplace.vocab):,}")
```

```
Laplace perplexity on held-out data: 4,895.5
Vocabulary size:                     73,808
```

A perplexity in this range against a vocabulary of tens of thousands of types means the model has learned something real — a uniform model over the vocabulary would have a perplexity equal to the vocabulary size. It is still a weak model in absolute terms, which is expected: add-one smoothing is the crudest option available, and it moves a lot of probability mass onto bigrams that genuinely never occur.

In [16]:
```
comparison = pd.DataFrame({
    "model": ["MLE (unsmoothed)", "Laplace (add-one)", "uniform baseline"],
    "held-out perplexity": [
        lm.perplexity(test_bigrams),
        laplace.perplexity(test_bigrams),
        float(len(laplace.vocab)),
    ],
})
comparison
```

Out[16]:

	model	held-out perplexity
0	MLE (unsmoothed)	inf
1	Laplace (add-one)	4.895489e+03
2	uniform baseline	7.380800e+04

---

5. Probability of the bigram *(pakistan is)*

$$P(\text{is} \mid \text{pakistan}) = \frac{\text{count}(\text{pakistan is})}{\text{count}(\text{pakistan})}$$

In [17]:
```
c_bigram = lm.counts[["pakistan"]]["is"]
c_context = lm.counts["pakistan"]

print(f"count('pakistan is') = {c_bigram:,}")
print(f"count('pakistan')    = {c_context:,}")
print(f"\nP(is | pakistan) by hand      = {c_bigram / c_context:.6f}")
print(f"P(is | pakistan) from the model = {lm.score('is', ['pakistan']):.6f}")
print(f"P(is | pakistan) with Laplace   = {laplace.score('is', ['pakistan']):.6f}")
```

```
count('pakistan is') = 616
count('pakistan')    = 12,150

P(is | pakistan) by hand      = 0.050700
P(is | pakistan) from the model = 0.050700
P(is | pakistan) with Laplace   = 0.007178
```

The hand calculation and `lm.score` agree, which confirms that `score` is doing the conditional count ratio and nothing else.

The Laplace estimate is noticeably smaller. Add-one smoothing inflates the denominator by the whole vocabulary size, so it takes probability away from bigrams that *were* observed and redistributes it to the ones that were not — the price paid for never returning zero.

In [18]:
```
followers = lm.counts[["pakistan"]]
top_followers = pd.DataFrame(
    [(w, c, lm.score(w, ["pakistan"])) for w, c in followers.most_common(10)],
    columns=["word after 'pakistan'", "count", "P(word | pakistan)"],
)
top_followers
```

Out[18]:

	word after 'pakistan'	count	P(word | pakistan)
0	</s>	2320	0.190947
1	is	616	0.050700
2	's	470	0.038683
3	and	415	0.034156
4	s	390	0.032099
5	has	285	0.023457
6	army	271	0.022305
7	cricket	244	0.020082
8	to	194	0.015967
9	from	179	0.014733

The single most likely thing to follow `pakistan` is `</s>` — the end-of-sentence marker — at around 19%, well ahead of `is`. That is a real property of the corpus rather than an artefact: `pakistan` frequently ends a tweet, either as a location tag or as the last word of a slogan. It is also a reminder that the padding symbols are ordinary vocabulary items to the model, and they compete for probability mass with real words.

---

6. Perplexity of the word *(pakistan)*

Perplexity of a single word is the inverse of its probability, $PP(w) = P(w)^{-1}$. Because the model was trained with `padded_everygram_pipeline`, it holds unigram counts as well as bigram counts, so a one-element n-gram is a valid input.

In [19]:
```
p_unigram = lm.score("pakistan")
print(f"P(pakistan)  = {p_unigram:.8f}")
print(f"PP(pakistan) = {lm.perplexity([('pakistan',)]):,.2f}")
print(f"Check, 1 / P = {1 / p_unigram:,.2f}")
```

```
P(pakistan)  = 0.00548268
PP(pakistan) = 182.39
Check, 1 / P = 182.39
```

The value is large, but that is the correct reading: on its own, with no context, `pakistan` is one guess among a vocabulary of tens of thousands, so the model is heavily "perplexed" by it. Context is what collapses that uncertainty — compare the same word once the preceding word is known:

In [20]:
```
rows = []
for context, word in [(None, "pakistan"), (["in"], "pakistan"), (["pakistan"], "is"),
(["pakistan"], "zebra")]:
    ngram = (word,) if context is None else (*context, word)
    score = lm.score(word, context)
    rows.append({
        "n-gram": " ".join(ngram),
        "probability": score,
        "perplexity": (1 / score if score else float("inf")),
    })

pd.DataFrame(rows)
```

Out[20]:

	n-gram	probability	perplexity
0	pakistan	0.005483	182.392675
1	in pakistan	0.064905	15.407200
2	pakistan is	0.050700	19.724026
3	pakistan zebra	0.000000	inf

Knowing the previous word cuts the perplexity by orders of magnitude — the entire point of moving from a unigram to a bigram model. And `pakistan zebra`, never seen in training, returns the zero/infinity pair that motivated smoothing in section 4.

Summary

1. **Loaded** ~202k tweets and filtered to the predominantly Latin-script ones, since a bigram model cannot usefully span two languages at once.
2. **Preprocessed** by stripping URLs, mentions, hashtags, `RT` markers and emojis, then split each tweet into sentences so that padding marks real boundaries.
3. **Built** a bigram MLE model with `padded_everygram_pipeline` and generated sample tweets — fluent between adjacent words, incoherent beyond that, exactly as a bigram model predicts.
4. **Evaluated** on a 10% held-out split: MLE gives infinite perplexity because a meaningful share of test bigrams were never seen in training. Laplace smoothing makes the number finite and well below the uniform baseline.
5. **P(is | pakistan)** computed from the model and verified by hand against the raw counts.
6. **PP(pakistan)** is the inverse of its unigram probability; adding one word of context reduces perplexity by orders of magnitude.